NLP hw2

Tamar Tabbach, Gal Barak, Adam Fleisher

May 2026

1 Word-Level Neural Bi-gram Language Model

(a)

Step 1: Simplifying the Cross Entropy Function

First, we substitute the definition of the softmax function into the Cross Entropy loss and apply logarithm rules:

$$\mathrm{CE}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_i y_i \log(\hat{y}_i)$$

$$= -\sum_i y_i \log\left(\frac{\exp(\theta_i)}{\sum_j \exp(\theta_j)}\right)$$

$$= -\sum_i y_i \left(\log(\exp(\theta_i)) - \log\left(\sum_j \exp(\theta_j)\right)\right)$$

Since $\log(\exp(x)) = x$, we get:

$$= -\sum_i y_i \left(\theta_i - \log\left(\sum_j \exp(\theta_j)\right)\right)$$

Step 2: Differentiating with respect to θ_k

Now we take the partial derivative with respect to θ_k:

$$\frac{\partial \mathrm{CE}}{\partial \theta_k} = -\frac{\partial}{\partial \theta_k}\left[\sum_i y_i\theta_i - \sum_i y_i \log\left(\sum_j \exp(\theta_j)\right)\right]$$

$$= -\left[y_k \cdot 1 - \sum_i y_i \cdot \frac{\partial}{\partial \theta_k} \log\left(\sum_j \exp(\theta_j)\right)\right]$$

Using the chain rule for the logarithm term:

$$= -\left[y_k - \sum_i y_i \cdot \frac{\exp(\theta_k)}{\sum_j \exp(\theta_j)}\right]$$

We can pull out the term that does not depend on i:

$$= -y_k + \left(\sum_i y_i\right) \frac{\exp(\theta_k)}{\sum_j \exp(\theta_j)}$$

Since $\mathbf{y}$ is a one-hot vector, the sum of its elements is 1 (i.e., $\sum_i y_i = 1$). Additionally, by definition, $\frac{\exp(\theta_k)}{\sum_j \exp(\theta_j)} = \hat{y}_k$. Substituting these back yields the final result:

$$= -y_k + 1 \cdot \hat{y}_k = \hat{y}_k - y_k$$

(b)

Step 1: Applying the Chain Rule

To find the gradient of the loss J with respect to the input $\mathbf{x}$, we use the chain rule. First, let us define the intermediate variables of the network:

$$\begin{aligned}
\mathbf{z}_1 &= \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1 \\
\mathbf{h} &= \sigma(\mathbf{z}_1) \\
\mathbf{z}_2 &= \mathbf{h}\mathbf{W}_2 + \mathbf{b}_2 \\
\hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}_2)
\end{aligned}$$

The chain rule backward from the loss J to the input $\mathbf{x}$ is given by:

$$\frac{\partial J}{\partial \mathbf{x}} = \frac{\partial J}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}_1} \cdot \frac{\partial \mathbf{z}_1}{\partial \mathbf{x}}$$

Step 2: Calculating Individual Derivatives

Now, we compute each component in the chain separately:

- $\frac{\partial \mathbf{z}_1}{\partial \mathbf{x}} = \mathbf{W}_1^T$ (Derivative of the first linear transformation)
- $\frac{\partial \mathbf{h}}{\partial \mathbf{z}_1}$ represents the derivative of the sigmoid function. It is structured as a $D_h \times D_h$ diagonal Jacobian matrix where the elements on the main diagonal are the derivatives of each individual activation h_i, and all off-diagonal elements are zero:

$$\begin{pmatrix}
h_1(1-h_1) & 0 & \cdots & 0 \\
0 & h_2(1-h_2) & \cdots & 0 \\
\vdots & \vdots & \ddots & \vdots \\
0 & 0 & \cdots & h_{D_h}(1-h_{D_h})
\end{pmatrix}$$

- $\frac{\partial \mathbf{z}_2}{\partial \mathbf{h}} = \mathbf{W}_2^T$ (Derivative of the second linear transformation)
- $\frac{\partial J}{\partial \mathbf{z}_2} = \hat{\mathbf{y}} - \mathbf{y}$ (This is the gradient of the Cross Entropy loss with respect to the input of the softmax function, which was already derived in section a)

Step 3: The Final Result
Multiplying all the terms together from left to right, we get the final gradient:

$$\frac{\partial J}{\partial \mathbf{x}} = (\hat{\mathbf{y}} - \mathbf{y})\mathbf{W}_2^T \begin{pmatrix} h_1(1-h_1) & 0 & \cdots & 0 \\ 0 & h_2(1-h_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & h_{D_h}(1-h_{D_h}) \end{pmatrix} \mathbf{W}_1^T$$

(d)

Based on the training iterations of the neural bi-gram language model using GloVe embeddings, the final evaluation yielded the following result:

- **Dev Perplexity:** 112.8621

Note: The trained parameters have been saved to `saved_params_40000.npy` *and are included in the submission zip file.*

2 Generating Shakespeare with a Char-level LM

(a)

Character-based models easily handle out-of-vocabulary words and misspellings by building them character by character, and they require far fewer embedding parameters. Meanwhile, word-based models capture semantic meaning and contextual relationships more naturally. Additionally, word-based models yield shorter sequences, reducing computational costs and making it easier for the network to learn long-range dependencies without forgetting past context.

(c)

3 Perplexity

(a)

Let $x_i = p(s_i|s_1, ..., s_{i-1})$ represent the conditional probability of token s_i. We begin with the left-hand side of the perplexity equation:

$$2^{-\frac{1}{M}\sum_{i=1}^{M}\log_2 x_i}$$

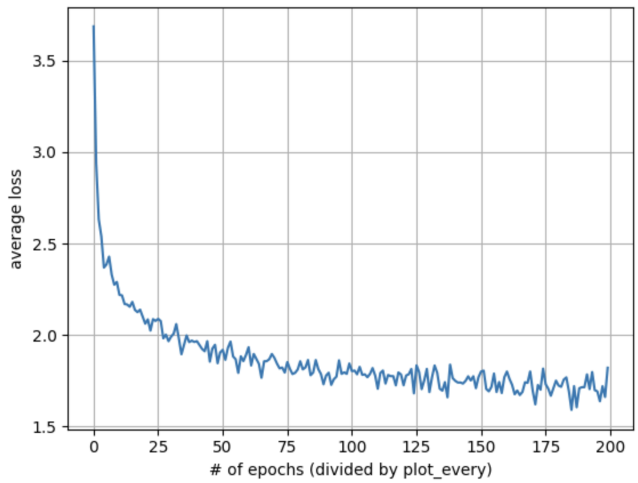

Figure 1: Training loss curve for the character-level Shakespeare language model: average loss versus number of epochs (sampled every `plot_every` steps).

Using the algebraic property of exponents where $a^{B \cdot C} = (a^B)^C$, we can isolate the summation term from the outer scaling factor:

$$= \left(2^{\sum_{i=1}^{M} \log_2 x_i}\right)^{-\frac{1}{M}}$$

Next, using the exponent property where a sum in the exponent translates to a product of bases ($a^{\sum z_i} = \prod a^{z_i}$), we can rewrite the inner expression as:

$$= \left(\prod_{i=1}^{M} 2^{\log_2 x_i}\right)^{-\frac{1}{M}}$$

Since $2^{\log_2 x_i} = x_i$, the terms inside the product simplify directly to the joint probability components:

$$= \left(\prod_{i=1}^{M} x_i\right)^{-\frac{1}{M}}$$

Next, using the natural logarithm identity, we can express each probability term x_i in base e as $x_i = e^{\ln x_i}$:

$$= \left(\prod_{i=1}^{M} e^{\ln x_i}\right)^{-\frac{1}{M}}$$

When multiplying terms with identical bases, we add their exponents together ($\prod e^{z_i} = e^{\sum z_i}$), turning the product back into a summation in the exponent of e:

$$= \left(e^{\sum_{i=1}^{M} \ln x_i}\right)^{-\frac{1}{M}}$$

Finally, by applying the power-of-a-power exponent rule once more, we multiply the inner and outer exponents:

$$= e^{-\frac{1}{M}\sum_{i=1}^{M} \ln x_i}$$

Substituting $x_i = p(s_i|s_1, ..., s_{i-1})$ back into the formula gives us the right-hand side of the equation:

$$= e^{-\frac{1}{M}\sum_{i=1}^{M} \ln p(s_i|s_1,...,s_{i-1})}$$

This completes the proof, showing that both expressions are mathematically identical.